

RECOVERY PLANNING UNDER FROZEN TOOL GRAPHS: A SMALL STRUCTURAL HOLDOUT COMPARING MODEL-REQUESTED RERUNS WITH DETERMINISTIC CLOSURE

Anonymous authors

ABSTRACT

We report `recovery_v2`, a small self-authored structural holdout that asks whether a model planner adds incremental recovery value over a deterministic actual-graph closure baseline when tool dependencies are already supplied. Nine held-out base instances span three task families (tabular, retrieval, classification) crossed with three shard topologies, each evaluated under four scenarios (clean, `aux_update`, `joint_update`, `incomplete_lineage`). One saved model plan per episode is shared by policies A, D, E, and F, yielding 36 plans and 180 policy replays; G is deterministic and consumes no model plan. All 180 replays completed correctly with zero wrong completions, zero refusals, and zero stale completions. $E - A$ was exactly zero on both correct completion and tool calls for every one of the nine paired base instances. F used 414 tool calls versus 294 for A, D, E, and G. Because the paired unit is the base instance and scenarios are repeated measures, equality in this design does not establish statistical equivalence or generalization. The protocol was frozen locally before the run but is not a public preregistration, and the scorer is a separate implementation by the same authors rather than external reproduction. We report these results as a bounded negative finding about incremental model planning under supplied dependencies, not as evidence of a novel dependency-discovery algorithm. Full replay using more calls does not establish incremental E gain.

1 INTRODUCTION

Tool-using agents that recompute derived artifacts after upstream data changes face a planning question: which nodes must be rerun, and in what order? A natural hypothesis is that a language model planner can improve recovery by reasoning about which cached results are invalidated. This study tests that hypothesis in a deliberately constrained setting where the relevant dependency information is already available to every policy and to the model. The research question is therefore narrow: given frozen graph generators, visible actual and declared dependencies, and a fixed scorer, does a model-requested recovery policy (E) outperform a model-requested rerun policy that preserves the original request order including duplicates (A) on correct completion or tool-call cost? We also report a deterministic actual-graph closure baseline (G) and a full-replay policy (F) so that any apparent E advantage can be compared against a non-model alternative and a conservative safe alternative. Full replay using more calls does not establish incremental E gain.

The question was motivated by earlier agent proposals about recovery behavior, but the protocol was corrected by the developer before freezing. Specifically, dependency propagation, the oracle definition, matched information across policies, and the structural split were revised. We disclose this intervention because it means the study is not an autonomous scientific artifact produced end to end by the model; it is a developer-assisted experiment whose question originated partly from agent proposals. We also distinguish native JiuwenSwarm orchestration, which executed the frozen protocol, from scientific autonomy: orchestration ran the specified plans and replays, but the protocol, corrections, and analysis framing were developer-assisted. Local Python tools executed and scored the CPU replays.

The main study is recovery_v2, not the historical development study. The historical run replay_runs/live-20260928T144804-532140 is retained as context, but its numeric results have not been verified by this adapter and are not part of the primary evidence. Historical development and post-hoc results are not primary v2 evidence and cannot isolate one cause of improvement. The contribution of this manuscript is a transparent report of a small, frozen, self-authored structural holdout in which the incremental benefit of model-requested closure over model-requested reruns was not observed.

2 RELATED WORK

Prior work on agent reliability and provenance provides context for the design, but we describe each cited source only within its recorded version, access scope, and supported findings. All cited statements are within the supplied reading notes' scope. AgentCheck (arxiv:2607.11098v1) presents a reproduce-intervene-mitigate workbench for LLM agents over MCP; its Algorithm 1 changes one cached tool response, and Section 6 explicitly excludes evaluation of compound and cascading failures. Its retry, schema, and filter wrappers are examined, but it is not a published dependency-invalidation baseline. We therefore do not treat it as a comparison method for our recovery policies.

Provenance integrity work (arxiv:2608.12761v1) notes that selective recovery already follows transitive declared dependencies after version changes, and that full rerun is a strong safe alternative when lineage is unavailable. It also observes that contracts and declared dependencies can themselves be incomplete. This is directly relevant to our incomplete_lineage scenario, where the declared graph is damaged while the actual tool graph remains intact. Our study does not claim to discover dependencies; it supplies actual dependencies to all policies and the model, so static actual-graph closure is established prior art rather than a novel mechanism.

AgentProp-Bench (arxiv:2604.16706v1) studies parameter injection and propagation with simulated tools. Its interceptor combines schema and error checks, uncertainty keywords, and numerical consistency, and the paper separates judge validation from task outcomes. Our design similarly separates numerical correctness from source freshness, and we do not reinterpret malformed plans as voluntary refusal. None of these sources is reproduced as a baseline here; they inform terminology and the interpretation of limitations only.

(Aritra Mazumder & Nusrat jahan Lia, 2026; Jesus Salas, 2026; Bhaskar Gurram, 2026).

3 METHODS

The study uses nine self-authored held-out base instances. Each instance is a frozen graph generator configuration with a task family (tabular, retrieval, classification) and a shard topology for two sources, main and aux. Held-out topologies are (main=2, aux=2), (main=3, aux=2), and (main=2, aux=3), giving three layouts per family. Seeds are explicit arithmetic identifiers, not random sampling: $1009 + \text{split_offset} + 101 * \text{family_index} + 17 * \text{layout_index}$, with split_offset 10000 for held-out. Each case has eight keys k00..k07. Tabular uses three positive transactions per key; retrieval uses revisions 2, 1, 3 deliberately unsorted; classification has one prediction and initially matching label per key. Source rows retain the replay_reference schema, and partitioning is performed by the executor.

The tool graph is fixed per topology. For main=2, aux=2, the nodes are read_main, partition_main_0, summarize_main_0, partition_main_1, summarize_main_1, summarize_main, read_aux, partition_aux_0, summarize_aux_0, partition_aux_1, summarize_aux_1, summarize_aux, combine, and report. For main=3, aux=2, main has three partitions and three summarize nodes merged into summarize_main. For main=2, aux=3, aux has three partitions and three summarize nodes merged into summarize_aux. Operations are read, partition, summarize, merge, combine, and report. Partitioning splits by key UTF-8 byte sum modulo shard count; each shard summarizes independently; merge combines non-overlapping keys; combine performs the domain computation; report copies the final structure. Every shard tool counts as one call, including empty shards. Tool counts exclude shared initial-cache construction and are not measured dollar savings.

Four scenarios are applied per instance. In clean, source data are unchanged and declared dependencies are complete. In aux_update, auxiliary source changes and declared dependencies remain

complete. In `joint_update`, both sources change and declared dependencies remain complete. In `incomplete_lineage`, the data are identical to `aux_update`, but only the declared `summarize_aux-to-combine` edge is removed; the actual tool graph is unchanged. Auxiliary updates affect `k00` by increasing its rate, toggling its enabled flag, or flipping its label. Joint updates additionally affect `k01` by adding seven to one transaction or the latest retrieval value, or by flipping its prediction. Source-level revisions increase only for changed sources.

Policies are defined as follows. A requests reruns in their original order, including duplicates. D requests the union of the model-requested nodes and the declared-graph closure of changed sources, then executes in actual topological order. E requests the union of the model-requested nodes and the actual-graph closure of changed sources, then executes in actual topological order. F performs full replay when sources change and deduplicated model requests on clean inputs. G performs deterministic actual-graph closure of changed sources and requires no model plan. A, D, E, and F share one saved model plan per episode; G has zero model requests and uses no saved model plan. The model was `deepseek-flash` with temperature 0 and reasoning disabled. The model outputs JSON actions, each rerun containing a node, ending with `emit` or `refuse`. Illegal JSON, illegal actions, and missing terminal actions are recorded and are not silently repaired into an `emit`. A, D, E, and F respect explicit refusal and structurally invalid plans; G is deterministic and does not fail because of model refusal. Malformed plans are not voluntary refusal.

The scorer is a separate `replay_reference.py` implementation by the same authors, which reads complete source rows directly and does not read the declared graph, execution cache, or policy gates. The executor and oracle are separate implementations by the same authors. Numerical correctness and provenance freshness are distinct outcomes and are reported separately. Freshness diagnostics record parent artifact hashes and source versions. The information condition supplies actual tool dependencies and declared dependencies to the planner and every policy; actual dependencies are supplied tool contracts, not discovered dependencies, and should not be labeled as discovered or automatically as answer leakage. The tool budget per scenario is twice the actual node count, allowing full replay to complete; model plans may request duplicate nodes within that budget. No transient failures were injected, so no retry-effectiveness claim is supported. Full replay using more calls does not establish incremental E gain.

The local pre-run freeze fixed the specification, source code, source data, and prompt snapshot before execution. This was a local freeze, not public preregistration, and the protocol was developer assisted. The paired analysis unit is the base instance, not the scenario or policy replay. Nine base instances, not policy executions, are paired units. There are 36 completed model plans and 180 completed policy replays, of which 144 are paired model-policy replays and 36 are deterministic G replays. The 36 plans and 180 executions are not 180 independent samples. The primary comparison is E minus A on correct completion and total tool calls; no gain is a valid outcome. We report E versus A and G as well as F. The strong baseline is G, which is actual graph closure without a model and is not a novel algorithm. E’s tool count cannot be lower than G’s for valid `emit` plans executed to completion under the stated implementation, including all 36 observed plans; invalid or `refuse` plans can make E stop at zero while G still executes, so this is not an unconditional theorem.

4 RESULTS

Rendering mode: live. A: Model-requested reruns in their original order, including duplicates. D: Model requests union declared-graph closure of changed sources; actual topological order. E: Model requests union actual-graph closure of changed sources; actual topological order. F: Full replay when sources change; deduplicated model requests on clean inputs. G: Deterministic actual-graph closure of changed sources; no model plan required.

Scenario-by-policy results

Scenario	Policy	n	Correct	Wrong	Noncompl.	Calls	Stale
clean	A	9	9	0	0	0	0
clean	D	9	9	0	0	0	0
clean	E	9	9	0	0	0	0
clean	F	9	9	0	0	0	0
clean	G	9	9	0	0	0	0
aux update	A	9	9	0	0	78	0
aux update	D	9	9	0	0	78	0
aux update	E	9	9	0	0	78	0
aux update	F	9	9	0	0	138	0
aux update	G	9	9	0	0	78	0
joint update	A	9	9	0	0	138	0
joint update	D	9	9	0	0	138	0
joint update	E	9	9	0	0	138	0
joint update	F	9	9	0	0	138	0
joint update	G	9	9	0	0	138	0
incomplete lineage	A	9	9	0	0	78	0
incomplete lineage	D	9	9	0	0	78	0
incomplete lineage	E	9	9	0	0	78	0
incomplete lineage	F	9	9	0	0	138	0
incomplete lineage	G	9	9	0	0	78	0

Paired base-instance results

Case	A	D	E	F	G	E-A correct	E-A calls
1	4/4; 30	4/4; 30	4/4; 30	4/4; 42	4/4; 30	0	0
2	4/4; 32	4/4; 32	4/4; 32	4/4; 48	4/4; 32	0	0
3	4/4; 36	4/4; 36	4/4; 36	4/4; 48	4/4; 36	0	0
4	4/4; 30	4/4; 30	4/4; 30	4/4; 42	4/4; 30	0	0
5	4/4; 32	4/4; 32	4/4; 32	4/4; 48	4/4; 32	0	0
6	4/4; 36	4/4; 36	4/4; 36	4/4; 48	4/4; 36	0	0
7	4/4; 30	4/4; 30	4/4; 30	4/4; 42	4/4; 30	0	0
8	4/4; 32	4/4; 32	4/4; 32	4/4; 48	4/4; 32	0	0
9	4/4; 36	4/4; 36	4/4; 36	4/4; 48	4/4; 36	0	0

Each policy cell is correct/episodes; calls, aggregated over the four scenarios. Differences are paired raw counts, not significance tests.

Case mapping: 1: recovery-v2-heldout-tabular-m2-a2-11009; 2: recovery-v2-heldout-tabular-m3-a2-11026; 3: recovery-v2-heldout-tabular-m2-a3-11043; 4: recovery-v2-heldout-retrieval-m2-a2-11110; 5: recovery-v2-heldout-retrieval-m3-a2-11127; 6: recovery-v2-heldout-retrieval-m2-a3-11144; 7: recovery-v2-heldout-classification-m2-a2-11211; 8: recovery-v2-heldout-classification-m3-a2-11228; 9: recovery-v2-heldout-classification-m2-a3-11245.

36 saved model plans are shared by A/D/E/F, yielding 144 model-policy replays. G requires zero model calls and has 36 separate deterministic executions. The 180 executions are not independent samples; 9 base instances are the paired units. Calls exclude shared initial-cache construction. Non-completion includes malformed plans and does not imply voluntary refusal. Stale counts completed outputs with outdated source provenance, separately from numerical correctness.

Actual tool dependencies and declared dependencies are visible to the planner and every policy. Actual dependencies are supplied tool contracts, not discovered dependencies.

The research question follows Agent proposals; the developer corrected dependency propagation, oracle definition, matched information, and structural split before this frozen experiment. Development fixtures are CPU checks, not model observations.

Study scope and limitations: Nine self-authored base instances are the paired analysis units; scenarios and policy replays are repeated measures. One saved model plan per episode is reused for A/D/E/F; G is deterministic and does not consume that plan. This is a small self-authored structural holdout, not independent external validation or a public benchmark. The local pre-run freeze is not public preregistration; the protocol was developer assisted. The reference scorer is a separate implementation by the same authors, not external independent reproduction. Numerical correctness and source freshness are separate outcomes; malformed plans are not voluntary refusal. Static

actual-graph closure is an existing mechanism; no novel dependency-discovery algorithm is established. Comparisons include E versus A and G as well as F; equality does not establish statistical equivalence or generalization. Tool-call counts exclude shared initial-cache construction and are not measured monetary or computational cost. No transient failures were injected; no retry-effectiveness claim is supported.

Recorded study resources: model calls: 36; total tokens: 113384; duration seconds: 47.758; initial cache tool calls: 138; cost: unavailable; Native study invocation only; excludes development, literature, and writing.

5 DISCUSSION

The primary comparison is E minus A. Across all nine paired base instances, E minus A was exactly zero on correct completion and exactly zero on tool calls. Both policies completed all 36 episodes correctly, with no wrong completions, no refusals, and no stale completions. The per-policy totals are 294 tool calls for A, D, E, and G, and 414 for F. The scenario-level pattern is consistent: clean scenarios used zero tool calls for all policies; `aux_update` and `incomplete_lineage` used 78 calls for A, D, E, and G and 138 for F; `joint_update` used 138 calls for all policies. These numbers show no incremental E benefit over A in this study. Selective-versus-full savings alone are not E-specific gains, because A, D, and G also used fewer calls than F in the scenarios where F replayed fully.

The deterministic G baseline is important for interpretation. G is deterministic, requires zero model calls, and uses no saved model plan. G used 294 tool calls, identical to A, D, and E, and completed all 36 episodes correctly. Because G uses actual-graph closure without any model plan, the observed tool-call totals do not demonstrate that model-requested closure adds value beyond deterministic closure. The protocol anticipated this: E's tool count cannot be lower than G's for valid emit plans executed to completion under the stated implementation, including all 36 observed plans, so no claim that E outperforms traditional closure is made. Invalid or refuse plans can make E stop at zero while G still executes, so this is not an unconditional theorem. Static actual-graph closure is established prior art, not a novel dependency-discovery algorithm. The actual dependencies were supplied to every policy and the model, so the study does not test discovery of unknown dependencies.

F's higher tool count reflects full replay when sources change. In `aux_update` and `incomplete_lineage`, F used 138 calls versus 78 for the other policies; in `joint_update`, all policies used 138 calls. This is consistent with F's definition rather than a defect. The `incomplete_lineage` scenario is notable because the declared graph is damaged while the actual graph is intact. A, D, E, and G all completed correctly with 78 calls, and F completed correctly with 138 calls. The data do not show a correctness penalty for any policy in this scenario, but the design is small and the scenario is a single structural perturbation.

Several limitations constrain interpretation. The study is a small self-authored structural holdout, not independent external validation or a public benchmark. The nine base instances are self-authored, and the paired unit is the base instance; nine base instances, not the 36 scenarios or 180 policy executions, are the paired units, and scenarios and policy replays are repeated measures. Equality in this small study does not establish statistical equivalence or generalization. The local pre-run freeze is not public preregistration, and the protocol was developer assisted. The reference scorer is a separate implementation by the same authors, not external independent reproduction. Numerical correctness and source freshness are separate outcomes; all freshness counts were current (36 current, 0 stale, 0 not completed for each policy), but this does not convert the correctness result into a broader reliability claim. Malformed plans are not voluntary refusal, and no malformed plans or refusals occurred in the recorded summaries. No transient failures were injected, so no retry-effectiveness claim is supported. Tool-call counts exclude shared initial-cache construction and are not measured monetary or computational cost. The historical prior study is not part of the primary evidence, and its numeric results have not been verified by this adapter. Historical development and post-hoc results are not primary v2 evidence and cannot isolate one cause of improvement.

The related work supports a cautious reading. AgentCheck (arxiv:2607.11098v1) excludes compound and cascading failures from its evaluation, so it does not provide a baseline for the multi-node recovery studied here, and it is not a published dependency-invalidation baseline. Provenance integrity work (arxiv:2608.12761v1) notes that selective recovery already follows transitive declared

dependencies and that full rerun is a strong safe alternative when lineage is unavailable; our F results are consistent with that framing. That work also observes that contracts and declared dependencies can themselves be incomplete, which is directly relevant to the incomplete_lineage scenario. AgentProp-Bench (arxiv:2604.16706v1) studies parameter injection and propagation with simulated tools and separates judge validation from task outcomes, which parallels our separation of numerical correctness and source freshness. None of these sources is reproduced as a baseline, and none is claimed to have been externally reviewed or accepted. All cited statements are supported within the supplied notes' scope.

A further limitation is that A, D, E, and F share one saved model plan per episode, while G is deterministic and requires zero model calls. The 36 plans and 180 executions are not 180 independent samples. The 144 paired model-policy replays reuse the same plans across policies, so policy differences reflect execution semantics rather than independent model generations. This design was chosen to isolate policy behavior under matched information, but it limits the strength of any inference about model planning variability. The model was deepseek-flash with temperature 0 and reasoning disabled, and the resource record reports 36 model calls, 113384 total tokens, and 47.758 seconds of native study invocation duration, excluding development, literature, and writing. These resource figures describe the run, not a cost-effectiveness claim. Tool-call counts exclude shared initial-cache construction and are not measured monetary or computational cost.

Finally, the study does not claim external validation, public benchmarking, or acceptance. The held-out structures were generated by the same developer-authored process as the development structures, and generating held-out structures does not constitute execution, external validation, or independently authored data. The results are best read as a bounded negative finding: under supplied dependencies and a frozen scorer, model-requested actual-graph closure did not improve correct completion or tool-call cost relative to model-requested reruns in original order, and it matched a deterministic closure baseline. The study does not support claims of retry effectiveness, broad robustness, algorithmic novelty, independent validation, or complete autonomy.

(Aritra Mazumder & Nusrat jahan Lia, 2026; Jesus Salas, 2026; Bhaskar Gurram, 2026).

6 CONCLUSION

In recovery_v2, a small self-authored structural holdout with nine paired base instances, E minus A was zero on both correct completion and tool calls for every instance. All 180 policy replays completed correctly, with no wrong completions, no refusals, and no stale completions. F used 414 tool calls versus 294 for A, D, E, and G. The deterministic G baseline matched A, D, and E on tool calls and correctness, so the data do not show incremental model-planning benefit over deterministic actual-graph closure. These results do not establish statistical equivalence or generalization, and they do not support claims about dependency discovery, retry effectiveness, or monetary savings. The study is a small self-authored structural holdout, not independent external validation or a public benchmark. The protocol was frozen locally before the run but is not public preregistration, and the scorer is a same-author separate implementation rather than external reproduction. Historical development and post-hoc results are not primary v2 evidence and cannot isolate one cause of improvement. We report the finding as a transparent negative result within its stated scope and note that the historical prior study remains unverified by this adapter. The study does not support claims of broad robustness, algorithmic novelty, independent validation, or complete autonomy.

REFERENCES

- Aritra Mazumder and Nusrat jahan Lia. AgentCheck: A Reproduce-Intervene-Mitigate Workbench for LLM Agents over MCP, 2026. <https://arxiv.org/abs/2607.11098v1>.
- Bhaskar Gurram. Evaluating Tool-Using Language Agents: Judge Reliability, Propagation Cascades, and Runtime Mitigation in AgentProp-Bench, 2026. <https://arxiv.org/abs/2604.16706v1>.
- Jesus Salas. Correct Is Not Governed: Provenance Integrity in Agentic Workflows, 2026. <https://arxiv.org/abs/2608.12761v1>.